



Tecnológico de Monterrey

Evidencia 1 - Actividad Integradora 1

Franco De Escondrillas Vázquez | A01739410

Octavio Hernández Loyo | A01739304

Ingeniería en Tecnologías Computacionales

Dr. Sandra Eugenia Barajas Montiel

TC2038

23 de septiembre, 2026

1. Introducción

En esta actividad analizamos dos transmisiones y tres códigos de referencia para resolver tres preguntas: si un código aparece dentro de una transmisión, cuál es el palíndromo más largo de cada transmisión y cuál es la subcadena común más larga entre ambas. Estas operaciones nos ayudan a reconocer coincidencias y regularidades en secuencias de caracteres. La coincidencia de un patrón indica su presencia en los datos aunque no necesariamente implica que existe un programa malicioso como un virus. La solución actual combina un arreglo de sufijos con su arreglo de prefijos comunes más largos (LCP) y el algoritmo de Manacher. Esta reflexión describe el código y la complejidad de los algoritmos.

2. Funciones implementadas y complejidades

Arreglo de sufijos y construcción del LCP

Con base en los algoritmos vistos en clase, desarrollamos la construcción del *Suffix Array* y la construcción del *LCP*, usando el algoritmo de Kasai para mantener una construcción de $O(n)$. Sin embargo, hicimos algunas modificaciones para que dependiendo del constructor que se utilice se pueda hacer búsqueda de patrones dentro de una transmisión, o se encuentre el *Longest Common Substring* entre dos transmisiones. Adicionalmente, se implementó el algoritmo SA-IS para ordenar el *Suffix Array* en $O(n)$ en lugar de $O(n \log_2 n)$.

Búsqueda de códigos en las transmisiones

La función `find` recorre el arreglo de sufijos y utiliza el *LCP* para conservar un prefijo ya comparado del patrón. Cuando encuentra coincidencias completas, conserva la menor posición en el texto original; si no hay ninguna, devuelve -1 . El contador `matches` no retrocede y avanza como máximo M veces para un patrón de longitud M . El recorrido tiene costo $O(N + M)$ por consulta después de construir la estructura, con $O(1)$ espacio auxiliar.

Palíndromo más largo en transmisión (Manacher)

Manacher nos ayuda en evitar la expansión desde el centro de cada índice a los lados como lo haría un acercamiento naive. Primero transforma una cadena de longitud N en otra de

longitud $2N + 1$, colocando un separador entre los caracteres y en los extremos, así es como podemos computar tanto palíndromos de longitud par e impar. El *radius* almacena la longitud de la expansión del palíndromo alrededor de cada elemento en nuestra cadena ya modificada.

Las variables *center* y *right* señalan el centro y el extremo derecho del palíndromo que llega más lejos. Al visitar i dentro de ese intervalo, se obtiene $mirror = 2 \times center - i$. El radio inicial se calcula como $\min(radius[mirror], right - i)$. La simetría permite reutilizar coincidencias comprobadas, mientras que el límite impide suponer que conocemos caracteres fuera del intervalo.

Aunque hay un ciclo de expansión dentro de un recorrido, el costo total es $O(n)$: las expansiones que extienden el intervalo hacen avanzar *right*, que nunca retrocede, y hay como máximo una comparación fallida por centro. La cadena transformada y los radios requieren $O(n)$ memoria. Recuperar el mejor resultado cuesta $O(1)$, porque su centro y longitud ya están calculados.

Visualización de la ejecución real

La visualización de Manacher utiliza un observador opcional que recibe el estado del mismo algoritmo de C++. *ManacherSession* controla el avance y transmite el estado por la entrada y salida estándar; el servidor local de Python conecta esa sesión con la página mediante HTTP. JavaScript recibe los datos en `updateState(data)` y actualiza la representación de los índices y radios.

La carga de archivos de la página utiliza *FileReader*. El usuario puede avanzar, reproducir, pausar y reiniciar el recorrido. La organización visual reúne la sesión, el servidor, la interfaz y el iniciador. Los tiempos de espera, las solicitudes y el dibujo de la página agregan costo a la demostración; no representan el tiempo de cómputo puro de Manacher.

3. Análisis de resultados entre distintos enfoques

Resumen del costo computacional

N representa la longitud del texto procesado, M la longitud de un patrón y L la longitud de las dos transmisiones concatenadas con sus delimitadores. El espacio indicado corresponde a cada operación o estructura; no se deben sumar varias veces los mismos arreglos compartidos.

Operación	Tiempo	Espacio
Construcción del arreglo de sufijos	$O(N \log N)$	$O(N)$
Construcción del LCP	$O(N)$	$O(N)$
Consulta find con estructura lista	$O(N + M)$	$O(1)$ auxiliar
Manacher completo	$O(N)$	$O(N)$
Consulta longestPalindrome	$O(1)$	$O(1)$
Recorrido lcs con estructura lista	$O(L)$	$O(1)$ auxiliar

Otros acercamientos

Para solo tres patrones por transmisión, KMP sería una alternativa con $O(N + M)$ por patrón sin el ordenamiento previo; no está implementado en esta versión. El arreglo de sufijos permite reutilizar una representación del texto y resolver también la subcadena común, aunque el método de construcción actual puede dominar el tiempo total.

Para palíndromos, expandir desde todos los centros puede costar $O(N^2)$ en cadenas repetitivas; Manacher reduce ese costo a $O(N)$ a cambio de almacenar los radios. Para la subcadena común, una alternativa de programación dinámica requiere $O(N_1 N_2)$ tiempo y puede usar $O(\min(N_1, N_2))$ espacio con filas reutilizadas. Estas comparaciones son teóricas: no se han medido tiempos entre implementaciones equivalentes.

4. Reflexión final

Franco De Escondrillas Vázquez | A01739410

Esta actividad final fue un buen elemento de contraste para poner a prueba y comparación los distintos algoritmos vistos durante el curso. En más de una ocasión, fue posible utilizar más de un algoritmo diferente para resolver las tres tareas que se piden en la actividad, sin embargo cada una tiene sus ventajas y desventajas, por lo que fue necesario pensar en el contexto de la actividad (solo dos transmisiones y tres patrones, pero con longitudes $|s|$ y $|p|$ extensas).

Dentro de las diferentes estrategias que se nos ocurrieron, decidimos optar por utilizar el *Suffix Array* para la búsqueda de patrones. Si bien un algoritmo como el *Suffix Tree* presenta una complejidad teórica menor, su factor constante de memoria y tiempo de ejecución hace que en la práctica, inicializar una de estas estructuras sea equivalente a hacer una búsqueda un poco menos optimizada con algún otro algoritmo. En un contexto en el que se cuenta con una única cadena original masiva s y muchas consultas pequeñas p , donde $|p| \ll |s|$, entonces hace mucho sentido utilizar una estructura como esta. Sin embargo, en el entorno al que nos enfrentamos reutilizar la estructura del *Suffix Array* ahorra tiempo de construcción y hace que el programa corra de forma limpia (código mejor organizado) y eficiente (por el bajo consumo de memoria de un SA).

Incluso, llegamos a considerar optimizar más nuestro método de búsqueda para los patrones de $O(n + m)$ (similar a KMP) a $O(m + \log_2 n)$ y utilizar el mismo *Suffix Array* para la búsqueda de palíndromos, en lugar de Manacher. Sin embargo, esto requería que utilizáramos una estructura de *Range Minimum Queries* con consultas de $O(1)$ y construcción de $O(n)$, para aprovechar el LCP compartido dentro de un rango para hacer una búsqueda binaria en el SA (en lugar de la búsqueda secuencial que se implementó en el find) y en cada uno de los centros del palíndromo (similar a Manacher, pero con LCP). Sin embargo, dicha estructura es muy compleja de programar y se sale de los temas vistos en el curso, por lo que decidimos conservar métodos eficientes, pero más sencillos.

Octavio Hernández Loyo | A01739304

En el transcurso de este módulo aprendí de manera visual y en código los distintos algoritmos de manejo de strings y al aplicarlos en este trabajo final fue una culminación del aprendizaje

de la clase. En lo personal, lo más interesante fue el planteamiento o planeación del trabajo, a la hora de escoger el acercamiento que se le iba a dar al trabajo, puesto que fue donde más investigación, aprendizaje extra, conversación e interpretación se dió. Fue ese esfuerzo extra, que usualmente no tengo que dar, lo que me llevó de este trabajo puesto que fue bastante recompensante y divertido hasta cierto punto.

No por eso menospreció el trabajo que hubo en la implementación, sin embargo ya se había hecho bastante trabajo previo por lo que no se complicó de sobremanera. Aun así nunca falla que en este tipo de trabajos existen los cambios a última hora ya sea para quitar o agregar funciones y uno debe acoplarse a la marcha para que se complete el trabajo y todos estén satisfechos con el resultado. No esperaba ciertos aspectos del proyecto, que pensé sería menos complicado de lo que resultó, esperó el resultado sea adecuado.

Algo que me gustaría resaltar fue un algoritmo que no se llegó a incluir por temas de complejidad de lógica y de explicar, que fueron las sparse tables y range minimum queries, mi compañero las planteó y me explicó su funcionamiento. Aunque no pudimos incluirlas en el trabajo me pareció muy interesante y vale la pena volverlo a estudiar en un futuro. En general una experiencia de aprendizaje muy completa.